

CS4031 – Compiler Construction

Course Project Proposal Spring 2026

Project Title: Durin's Code

Theme: 8-Bit Adventure

Team Members:

Student Name	Student ID
1. Abdul Moiz Hussain	23K-0553
2. Muhammed Abdullah Khan	23K-0607
3. Huzaifa Abdul Rehman	23K-0782

1. Language Concept

Durin's Code is a DSL created to compile interactive text adventures into playable game engines. It enables the definition of rooms, items, and NPCs through a structured syntax, removing the need for manual imperative coding. The compiler's primary objective is to transform world specifications into a manageable memory model that tracks player inventory and location in real-time. Inspired by the 8-bit era, this project showcases a complete compilation pipeline, ensuring that every game element is semantically verified before execution.

2. Key Features

- World & Entity Specification:** A unified declarative syntax to define the game's static universe, including Rooms, Items, and NPCs. This module handles the registration of entity properties (e.g., item damage, NPC health) into the global Symbol Table for semantic validation.
- Reactive Action System:** A conditional logic engine that maps player string inputs to state-changing events. It supports nested if/else blocks to evaluate the World

State (e.g., checking if an NPC is dead or an item is in the inventory) before executing commands.

3. **Persistent State Management:** A dedicated Memory Model that tracks mutable player attributes, such as location, health, and a dynamic inventory list. This ensures that changes made in one room (like dropping an item) persist throughout the execution.
4. **Static Semantic Analysis:** The compiler performs rigorous checks to ensure all exits lead to existing rooms and all actions reference valid items or NPCs, preventing runtime crashes in the generated game.
5. **Bytecode Generation & CLI Execution:** Instead of just interpreting script, the compiler produces a standalone executable format (.json bytecode). This is paired with a Terminal-based Runtime Environment that loads the compiled file and handles the interactive game loop.

3. Example Program

A code snippet with two rooms, items, and an NPC:

```
// World Definition

room "bag_end" {

    description "The cozy hole of a Hobbit. A golden ring glints on the table."

    item the_ring { power: 100, type: "artifact" }

    exit east "buckland"

}

room "mount_doom" {

    description "The air is thick with ash. The fiery Cracks of Doom loom ahead."

    npc sauron { health: 999, hostile: true }

    exit west "buckland"

}

// Action Handlers
```

```

action "take ring" {
    if current_room == "bag_end" {
        player.inventory += the_ring
        print "The Precious is yours."
    }
}

action "destroy ring" {
    if current_room == "mount_doom" && player.has_item(the_ring) {
        remove the_ring
        print "The ring is consumed by fire! Middle-earth is saved."
        player.win = true
    } else {
        print "You cannot do that here."
    }
}

```

4. Target Output

- **JSON Bytecode Generation** : The compiler transforms the source script into a structured JSON bytecode file. This acts as the final compiled product, containing the serialized world-graph (rooms, items, NPCs) and validated logic, successfully decoupling the language from the execution engine.
- **Virtual Machine & Runtime Runner** : A dedicated Virtual Machine (VM) serves as the terminal-based console. It loads the JSON bytecode, initializes the Global State Heap to track player inventory and health, and runs the interactive game loop to execute the compiled logic.
- **Three-Address Code**: Before final code generation, the compiler produces an Intermediate Representation (IR) in Three-Address Code (TAC). This flattens complex conditions into a machine-ready sequence of operations, enabling optimization and logic verification.
- **CLI Compiler Tool**: A command-line utility that manages the full pipeline. It includes an `-o` flag for bytecode generation and a `--debug` mode to output the Symbol Table and AST, ensuring full transparency of the compilation phases.

5. Expected Challenges

Challenge 1: Complex State Management

Managing mutable game state across multiple entities (rooms, items, NPCs, player) while maintaining consistency. Changes to one entity must properly cascade (e.g., removing a defeated NPC must also remove it from the room inventory).

Challenge 2: Semantic Validation

Ensuring all cross-references are valid (rooms referenced in exits exist; NPCs/items can only be in valid locations; actions reference valid entities). Building a comprehensive symbol table for all world entities.

Challenge 3: Action Matching & Conditional Execution

Parsing complex action handlers with nested conditionals and state checks. The interpreter must correctly evaluate conditions based on current game state and execute the appropriate action sequence.

Challenge 4: Scope and Variable Binding

Managing local vs. global scope (player properties vs. NPC properties vs. room properties). Properly resolving identifiers in action handlers to the correct entity.

Challenge 5: Error Recovery & Reporting

Providing meaningful error messages when users attempt invalid actions (e.g., taking items that do not exist, moving to non-existent rooms). Clear feedback on parse errors in the game definition.